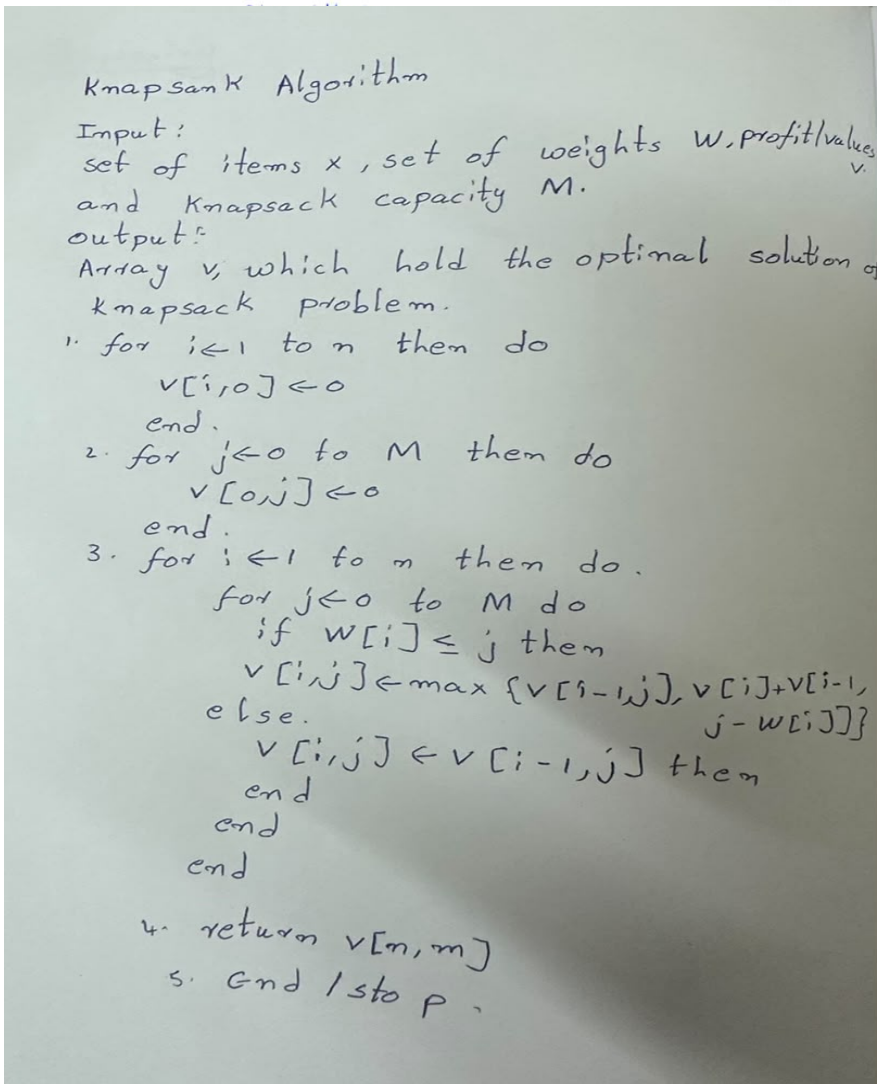
Practical: 5	Implementation of a knapsack problem using dynamic programming.

Aim: To implement the Knapsack Problem using Dynamic Programming and determine the maximum total value that can be obtained by selecting items without exceeding the given knapsack capacity.

Algorithm:



Knapsank Algorithm

Input:
set of items x, set of weights w, profit/values v.
and Knapsack capacity M.
output:
Array v, which hold the optimal solution of knapsack problem.

1. for i←1 to n then do
   v[i,0]←0
   end.
2. for j←0 to M then do
   v[0,j]←0
   end.
3. for i←1 to n then do.
   for j←0 to M do
   if w[i] ≤ j then
   v[i,j]←max {v[i-1,j], v[i]+v[i-1, j-w[i]]}
   else.
   v[i,j] ←v[i-1,j] then
   end
   end
   end
4. return v[n,m]
5. End /stop.

Program:

```
#include <iostream>
#include <algorithm>
using namespace std;

class Knapsack {
```

```
private:
    int w[10];          // weights
    int v[10];          // values
    int n;              // number of items
    int W;              // capacity
    int dp[10][100];    // DP table

public:

    void getInput() {
        cout << "Enter number of items : ";
        cin >> n;

        cout << "Enter knapsack capacity : ";
        cin >> W;

        for (int i = 1; i <= n; i++) {
            cout << "Item " << i << " weight and value : ";
            cin >> w[i] >> v[i];
        }
    }

    void solve() {

        for (int i = 0; i <= n; i++)
            dp[i][0] = 0;

        for (int j = 0; j <= W; j++)
            dp[0][j] = 0;

        // Fill DP table
        for (int i = 1; i <= n; i++) {

            for (int j = 1; j <= W; j++) {

                if (w[i] <= j) {

                    // Take or don't take item i
                    dp[i][j] = max(
                        v[i] + dp[i - 1][j - w[i]],
                        dp[i - 1][j]
                    );

                } else {


                    dp[i][j] = dp[i - 1][j];
                }
```

```
        }
    }
}

void printTable() {

    cout << "\nDP Table:" << endl;

    cout << "     ";

    for (int j = 0; j <= W; j++)
        cout << j << " ";

    cout << endl;

    for (int i = 0; i <= n; i++) {

        cout << "i=" << i << "  ";

        for (int j = 0; j <= W; j++)
            cout << dp[i][j] << " ";

        cout << endl;
    }
}

void findItems() {

    cout << "\nSelected Items:" << endl;

    int j = W;

    for (int i = n; i >= 1; i--) {

        if (dp[i][j] != dp[i - 1][j]) {

            cout << "Item " << i
                 << " (w=" << w[i]
                 << ", v=" << v[i]
                 << ") TAKEN" << endl;

            j = j - w[i];

        } else {

            cout << "Item " << i
                 << " (w=" << w[i]
                 << ", v=" << v[i]
```

```
                << ") NOT taken" << endl;
            }
        }
    }

    void showResult() {

        cout << "\nMaximum Value = "
            << dp[n][W] << endl;
    }
};

int main() {

    Knapsack k;

    k.getInput();
    k.solve();
    k.printTable();
    k.showResult();
    k.findItems();

    return 0;
}
```

Output:

```
Enter number of items : 4
Enter knapsack capacity : 5
Item 1 weight and value : 2
3
Item 2 weight and value : 3
4
Item 3 weight and value : 4
5
Item 4 weight and value : 5
6

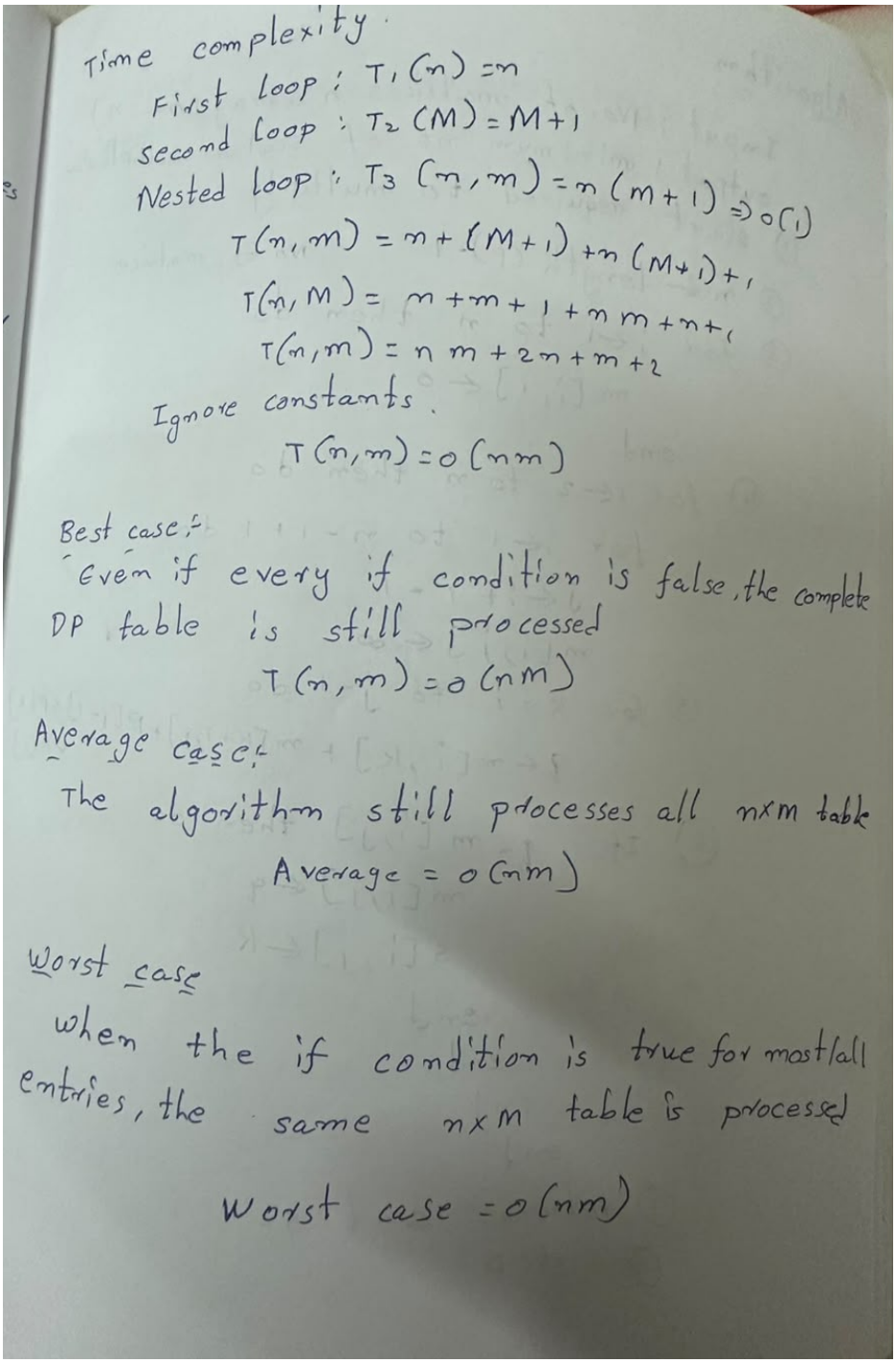
DP Table:
     0 1 2 3 4 5
i=0  0 0 0 0 0 0
i=1  0 0 3 3 3 3
i=2  0 0 3 4 4 7
i=3  0 0 3 4 5 7
i=4  0 0 3 4 5 7

Maximum Value = 7

Selected Items:
Item 4 (w=5, v=6) NOT taken
Item 3 (w=4, v=5) NOT taken
Item 2 (w=3, v=4) TAKEN
Item 1 (w=2, v=3) TAKEN

--------------------------------
Process exited after 35.36 seconds with return value 0
Press any key to continue . . .
```

Time Complexity:



Conclusion: The 0/1 Knapsack Problem was successfully implemented using the Dynamic Programming approach. The algorithm efficiently calculates the maximum value that can be obtained within the given weight capacity by storing and reusing previously computed results. This approach reduces redundant calculations and provides an optimal solution to the problem.